Assignment 3 - Autoencoder (AE)

Group 15

UD24016	Sezal Rana
S25016	Mohit Kulhari
B24036	Madhav Aggrawal
B24332	Anoushka Waghmare
B24102	Aashvi Garg
B23336	Akshat Jha
B23373	Harsh Vardhan Sharma
B23367	Bhavya Sunkari
B23387	Shubhankit Singh

March 18, 2026

1 The Bottleneck Challenge (Undercomplete AE)

In this part, we focus on understanding how an autoencoder compresses high-dimensional image data into a compact latent space & then reconstructs it:

- First, we implement a symmetric Autoencoder for MNIST with encoder architecture 784 $\rightarrow$ 128 $\rightarrow$ 64 $\rightarrow$ N and decoder architecture N $\rightarrow$ 64 $\rightarrow$ 128 $\rightarrow$ 784.
- Then same model is trained using two different bottleneck sizes, N=2 and N=32, to study how latent-space size affects reconstruction quality.
- Finally we compare a few original vs. reconstructed digit images for both bottleneck settings to visually examine the effect of stronger vs. weaker compression.

1.1 Dataset

The MNIST dataset consists of 28×28 grayscale images of handwritten digits (0–9), with 60,000 training and 10,000 test samples. Each image is flattened to a 784-dimensional vector and pixel values are normalised to $[0, 1]$ via `transforms.ToTensor()`.

1.2 Methodology

Component	Layer	Dimensions	Activation
Encoder	Linear 1	$784 \to 128$	ReLU
	Linear 2	$128 \to 64$	ReLU
	Linear 3	$64 \to N$	Linear
Decoder	Linear 4	$N \to 64$	ReLU
	Linear 5	$64 \to 128$	ReLU
	Linear 6	$128 \to 784$	Sigmoid

Table 1: Autoencoder architecture. N denotes the bottleneck size.

The autoencoder was trained using the Adam optimizer with a learning rate of 1×10^{-3}, batch size 128, and 30 epochs. Binary Cross-Entropy (BCE) loss was used since the input pixel values lie in $[0, 1]$ and the final output layer uses a Sigmoid activation, making BCE suitable for reconstruction. Also, ReLU activations were applied after the hidden linear layers in both the encoder and decoder to introduce non-linearity, while the bottleneck layer remained linear to learn the latent representation.

1.3 Results

1.3.1 Reconstruction Loss Curves

Figure 1 shows the reconstruction loss over 30 epochs for both configurations. By epoch 30, $N = 32$ achieved a lower loss (66.56) than $N = 2$ (134.15), indicating better reconstruction with a larger bottleneck.

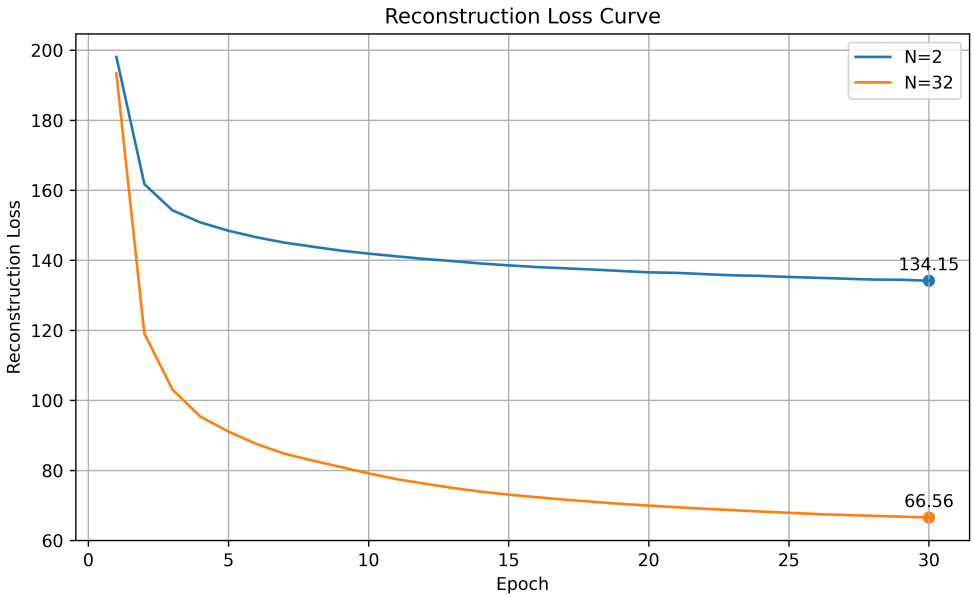

Figure 1: Reconstruction loss (BCE, sum-normalised) over 30 training epochs for $N = 2$ (blue) and $N = 32$ (orange). Final values are annotated.

1.3.2 Visual Reconstruction Comparison

Figure 2 shows eight test images alongside their reconstructions from both models.

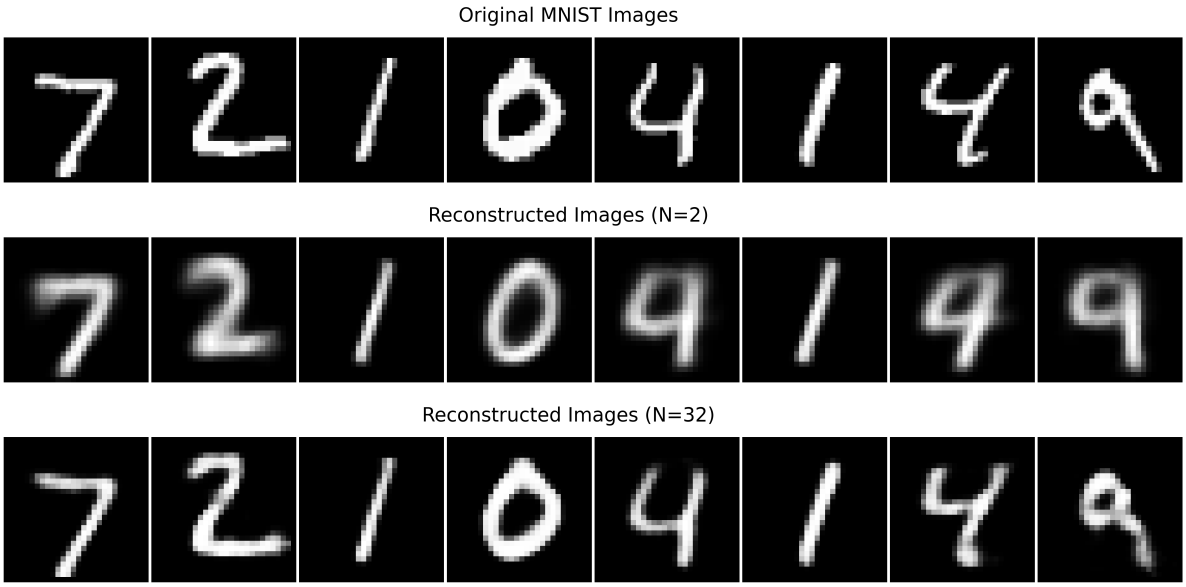

Figure 2: Top: original MNIST test images. Middle: reconstructions from $N = 2$. Bottom: reconstructions from $N = 32$.

1.4 Analysis

1.4.1 Effect of Bottleneck Size

$N = 32$ **(Moderate Compression):** With 32 latent dimensions the autoencoder retains sufficient capacity to encode per-digit stroke details. Reconstructions are visually sharp and closely resemble the originals, preserving fine features such as the inner loop of "0", the cross-stroke of "4", and the curved tail of "9". The final loss of **66.56** confirms strong quantitative performance.

$N = 2$ **(Extreme Compression):** Compressing a 784-dimensional image into just two numbers forces the encoder to discard most discriminative information. Reconstructions are blurry and over-smoothed. Structurally similar digits are conflated — for instance, the digit "4" is reconstructed as "9" (visible in Figure 2) because the two-dimensional code cannot distinguish them. The final loss of **134.15** — roughly twice that of $N = 32$ — corroborates this degradation.

1.4.2 Convergence Behaviour

The $N = 32$ model converges significantly faster. Both models begin at similar initial losses ($\approx$ 195–200), but by epoch 5, $N = 32$ has already dropped below 100 while $N = 2$ remains above 150. A larger bottleneck provides more gradient pathways, making optimisation easier.

1.4.3 Why Do the Reconstructions Differ?

1. **Information bottleneck:** A smaller N limits the information channel between encoder and decoder; information exceeding this capacity is irreversibly lost.

2. **Manifold hypothesis:** MNIST digits lie on a manifold whose intrinsic dimensionality far exceeds 2. Projecting onto R^2 causes class overlap and detail loss.

3. **Decoder over-smoothing:** With ambiguous latent codes, the decoder learns to output the *average* appearance of similar digits, yielding blurry reconstructions.

4. **Capacity vs. fidelity:** $N = 2$ barely distinguishes between digit classes, let alone preserving intra-class variation. $N = 32$ retains both.

2 Denoising and Feature Robustness

2.1 Objective

The goal of this experiment is to test how well a Denoising Autoencoder (DAE) can learn useful features from corrupted images. A standard Autoencoder learns to compress and reconstruct clean inputs. A Denoising Autoencoder (DAE) is trained differently: the input is corrupted with noise, but the model must reconstruct the original clean image. This forces the encoder to capture meaningful structural features rather than memorising pixel values.

2.2 Dataset

We used the Tiny-ImageNet-10 dataset for this experiment. The images are RGB and resized to 64×64. The dataset contains different object categories(spiders, frogs, turtles, goldfish, etc.), which helps in checking if the model can generalize well across variations. Split: 85% train / 15% val.

2.3 Methodology

2.3.1 Noise Injection

We added Gaussian noise to the input images before passing them to the encoder. The noisy image is defined as:

$$x_{noisy} = x + \epsilon, \quad \epsilon \sim \mathcal{N}(0, \sigma^2)$$

Here, x is the original image and ϵ is random noise. This step simulates real-world corruption and makes the task harder for the model.

A single-channel noise map is broadcast across all 3 RGB channels to produce natural grayscale grain (not colour distortion). Result clipped to [0, 1]

2.3.2 Model Architecture

We used a Convolutional Autoencoder (CAE) with two main parts:

- **Encoder:** A series of Conv2D layers with stride 2 that reduce the spatial size and extract features.
- **Latent Space:** A compressed feature representation that stores important information about the image.
- **Decoder:** ConvTranspose2D layers that reconstruct the image back to its original size.

  Each block uses Conv/ConvTranspose → BatchNorm → ReLU. Final decoder layer uses Sigmoid to constrain output to [0, 1]

Block	Operation	Output Shape
Encoder 1	Conv2d(3→32, s=2) + BN + ReLU	(B, 32, 32, 32)
Encoder 2	Conv2d(32→64, s=2) + BN + ReLU	(B, 64, 16, 16)
Encoder 3	Conv2d(64→128, s=2) + BN + ReLU	(B, 128, 8, 8)
Bottleneck	Latent representation	(B, 128, 8, 8)
Decoder 1	ConvTranspose2d(128→64, s=2) + BN + ReLU	(B, 64, 16, 16)
Decoder 2	ConvTranspose2d(64→32, s=2) + BN + ReLU	(B, 32, 32, 32)
Decoder 3	ConvTranspose2d(32→3, s=2) + Sigmoid	(B, 3, 64, 64)

Table 2: CAE Architecture

2.3.3 Loss Function

We trained the model using Mean Squared Error (MSE) loss:

$$\mathcal{L} * MSE = \frac{1}{N} \sum *i = 1^N (x_i - \hat{x}_i)^2$$

Here, x is the clean image and $\hat{x}$ is the reconstructed image.

2.3.4 Hyperparameter Tuning & Architecture Design

Two key design decisions were refined through experimentation:

Table 3: Hyperparameter Tuning Summary

Parameter	Tried	Final Choice	Reason
Noise σ	0.20, 0.10, 0.05	0.1	Lower σ = lighter corruption = sharper reconstruction
Noise channels	3-channel (`randn_like`)	1-channel broadcast	Eliminates random colour distortion in noisy images
Bottleneck	4×4 (4 encoder blocks)	8×8 (3 encoder blocks)	Less compression → sharper output
Noise type	'both' (Gaussian + S&P)	'gaussian' only	Double corruption made task too hard; single noise is sufficient

2.3.5 Training Details

- Optimizer: Adam
- loss: MSE
- LR: 0.001
- Number of epochs: 30
- Batch size: 64
- noise: Gaussian, sigma = 0.05

2.4 Results

2.4.1 Reconstruction Trio

Figure 3 shows the results of the model. Each column includes original clean image, noisy input image and denoised output from the model.

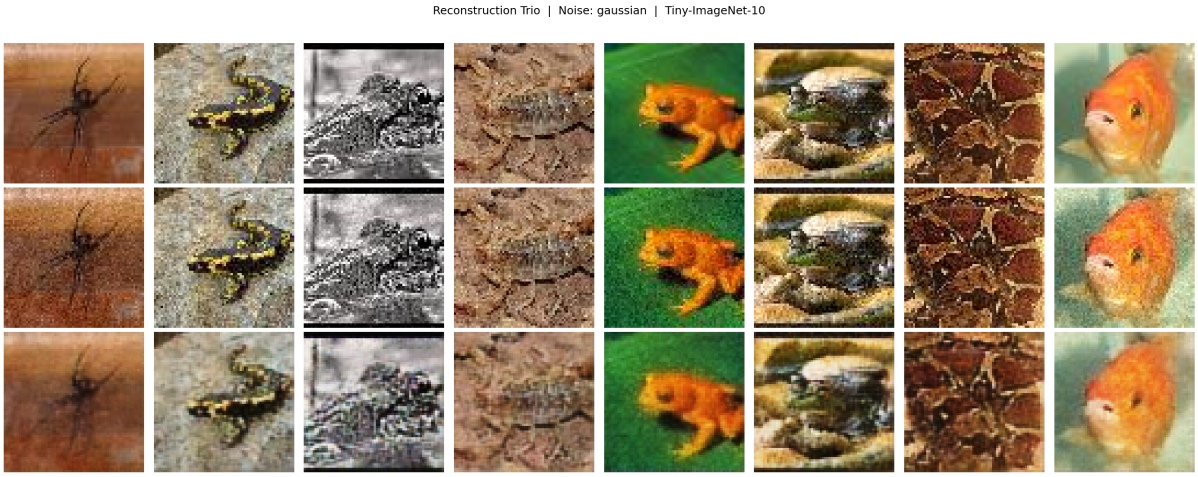

Figure 3: Reconstruction Trio: Original, Noisy Input, and Denoised Output

The model successfully recovers the main structure, colour, and content from noisy inputs across all 10 classes. Fine details such as spider legs (col 1), salamander spots (col 2), and goldfish fins (col 8) are visible in the denoised output. Mild blurriness is present and is expected from MSE loss (see 2.6 Analyis).

2.4.2 Reconstruction Loss Curve (30 Epochs)

The MSE loss was tracked over 30 epochs for train and validation sets. Loss dropped steeply in epochs 1–8 (rapid learning of coarse structure), then converged gradually through epoch 30. Train and validation loss remained close throughout, confirming no overfitting.

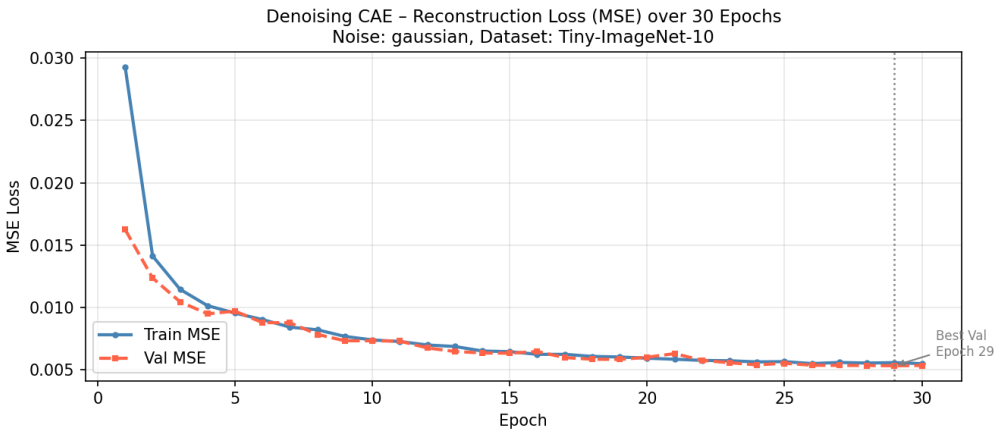

Figure 4: Reconstruction Loss (MSE) over 30 Epochs : Train and Validation

Table 4: Validation Set MSE by Noise Type (Best Model)

Noise Type	**MSE**
Gaussian noise	0.007223
Salt & Pepper noise	0.006731
Combined noise	0.007857

2.5 Quantitative Evaluation

The model achieves consistent MSE across all noise types: Gaussian (0.007223), Salt & Pepper (0.006731), and combined noise (0.007857). The slightly lower MSE on Salt & Pepper noise is expected, as the corruption probability is low (`SP_PROB = 0.02`), meaning only 2% of pixels are affected, a simpler denoising task than Gaussian noise which corrupts every pixel. Overall, the model generalises reasonably well to unseen noise types, with only a modest increase in MSE on combined noise (0.007857), suggesting it has learned general denoising features rather than simply memorising Gaussian patterns.

2.6 Analysis

The model removes most of the noise from the input images, which shows that it has learned useful features. The main structure of the objects is still visible in the reconstructed images.
However, the outputs look blurry compared to the original images. This happens because MSE loss works at the pixel level. It pushes the model to reduce average error, which leads to smooth images instead of sharp ones. As a result, fine details like edges and textures are lost.
Another reason for this is the compression in the latent space. When the image is compressed too much, some information is lost. The decoder then tries to rebuild the image from limited information, which leads to further smoothing.
Overall, the model performs well in removing noise but struggles to preserve fine details.

3 Anomaly Detection (Tiny-ImageNet-10)

3.1 Question 3.1: The 2D Manifold (Latent Mapping)

3.1.1 Objective

The objective of this experiment is twofold: first, to evaluate how an Autoencoder maps high-dimensional data (MNIST images) into a highly compressed, 2-dimensional bottleneck manifold to observe if classes naturally cluster by structural similarity. Second, to experiment with different activation functions (Sigmoid vs. Tanh) at the final output layer to determine which performs better when input pixels are normalised to the $[0, 1]$ range.

3.1.2 Methodology

- **Latent Extraction:** A trained MNIST Autoencoder with a bottleneck of $N = 2$ is used. The entire test set is passed exclusively through the Encoder to extract a 2D coordinate pair for each image.
- **Visualisation:** A scatter plot is generated where each 2D point represents an image, colour-coded by its actual digit label (0–9).
- **Activation Experiment:** Two identical Convolutional Autoencoders are trained using Mean Squared Error (MSE) loss. The only difference is the final Decoder layer: one uses a `Sigmoid` activation, and the other uses a `Tanh` activation.
- **Preprocessing:** Input images are normalised strictly to the $[0, 1]$ range using `ToTensor()`.

3.1.3 Results

The Autoencoder successfully compressed the test set into a 2D space, grouping the digits into distinct clusters as visualised in Figures 5 and 6.

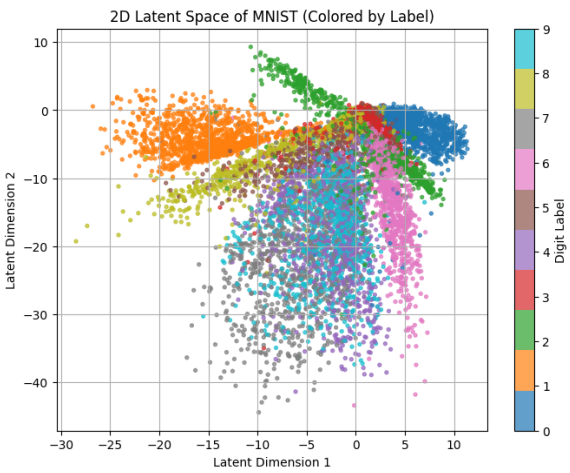

Figure 5: 2D Latent Space of MNIST (Coloured by Label)

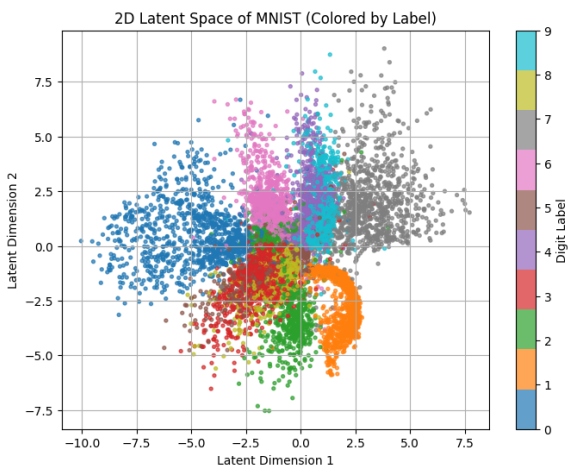

Figure 6: 2D Latent Space of MNIST (Alternative View)

Furthermore, the training loss for both the Sigmoid and Tanh configurations was recorded to evaluate reconstruction performance. The reconstruction loss curves are shown below in Figures 7 and 8.

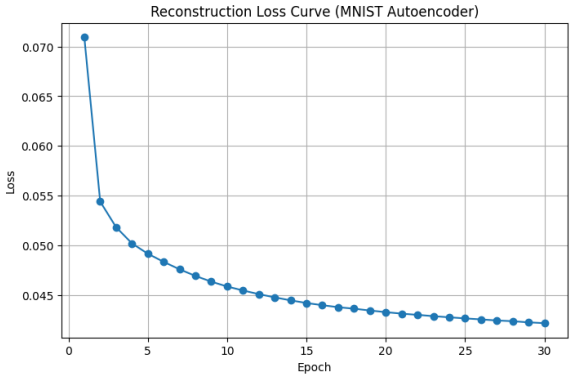

Figure 7: Reconstruction Loss Curve (Sigmoid Output)

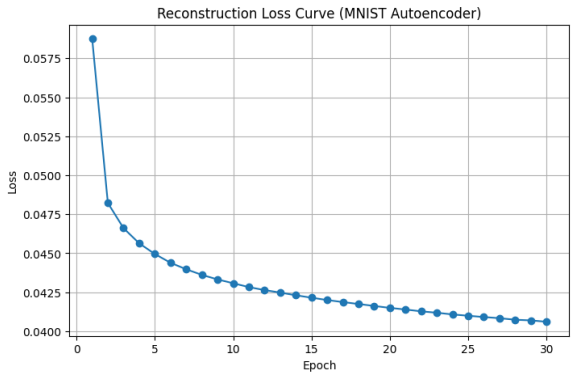

Figure 8: Reconstruction Loss Curve (Tanh Output)

3.1.4 Analysis

The results from the visualisations and loss curves reveal several key insights:

1. **Structural Grouping in Latent Space:** The scatter plots indicate that the network successfully groups similar digits together without being provided any labels during training. Digits with highly unique continuous structures, like '0', form isolated clusters.

2. **Distribution Overlap:** Because the bottleneck is restricted to only 2 dimensions, perfect separation is impossible. Digits with shared structural strokes overlap heavily. For example, '1' and '7' share diagonal/vertical lines and cluster closely together. Similarly, '4' and '9' exhibit heavy blending, as a '9' is visually interpreted as a '4' with a closed top.

3. **Sigmoid vs. Tanh (Reconstruction Loss):** As seen in the reconstruction loss graphs, the **Sigmoid** activation (Figure 7) is the superior choice. When inputs are normalised to exactly $[0, 1]$, Sigmoid mathematically bounds its output to $(0, 1)$, meaning the network's output distribution perfectly matches the target distribution, resulting in a smooth, stable loss curve.

4. **Capacity Waste with Tanh:** The **Tanh** function (Figure 8) bounds its output to the $(-1, 1)$ range. Because our target pixels never drop below 0, the network is forced to ignore the entire negative half of the Tanh activation space. This restricts gradient flow and forces the network to aggressively avoid negative outputs, making Tanh a suboptimal choice unless the inputs are explicitly re-normalised to $[-1, 1]$.

3.1.5 Conclusion

The Autoencoder effectively learns a structurally meaningful 2D manifold, successfully grouping digits by their morphological features despite the severe $N = 2$ bottleneck. Overlap occurs predictably among visually similar digits. Furthermore, the reconstruction loss analysis confirms that the Decoder's output function must align with the input normalisation scheme; Sigmoid is the correct and most efficient choice for inputs scaled to $[0, 1]$.

3.2 Objective

The objective of this experiment is to evaluate whether a trained Autoencoder can be used for anomaly detection using reconstruction error. Since Tiny-ImageNet-10 contains anonymised class labels, one class is treated as an "Unseen" (anomalous) class. The Autoencoder is trained exclusively on the remaining 9 "Seen" classes, thereby learning their underlying data distribution. At inference time, images from both Seen and Unseen classes are passed through the network to analyse differences in reconstruction behaviour.

3.2.1 Methodology

- **Data Split:** Class `n12267677` is treated as the Unseen (anomalous) class. The model is trained only on the remaining 9 Seen classes.
- **Preprocessing:** Input images are normalised to the range $[0, 1]$ using `ToTensor()`. The model uses a **Sigmoid** activation at the output layer, which naturally produces outputs in $[0, 1]$, directly consistent with the input domain. No additional rescaling is required.
- **Architecture & Training:** A Convolutional Autoencoder (CAE) using `Conv2d` and `ConvTranspose2d` layers is trained using the Adam optimizer for 30 epochs with Mean Squared Error (MSE) loss.
- **Anomaly Score:** For a given input x, the anomaly score is defined as:

$$s(x) = \frac{1}{N} \sum_{i=1}^{N} (x_i - \hat{x}_i)^2$$

  where $\hat{x}$ is the reconstructed output. Higher values of $s(x)$ indicate poorer reconstruction and a higher likelihood of the sample being anomalous.
- **Evaluation:** Reconstruction errors are computed for both Seen and Unseen test samples, and their distributions are compared via a density histogram.

3.2.2 Results

The model converged successfully over 30 epochs. The average reconstruction error on the test data is summarised below:

Table 5: Average Reconstruction Error (MSE) on Test Data (Sigmoid)

Data Group	**Average MSE**
Seen Classes (Normal Data)	0.0096
Unseen Class (Anomaly Data)	0.0122

Figure 9 shows the distribution of reconstruction errors for both groups.

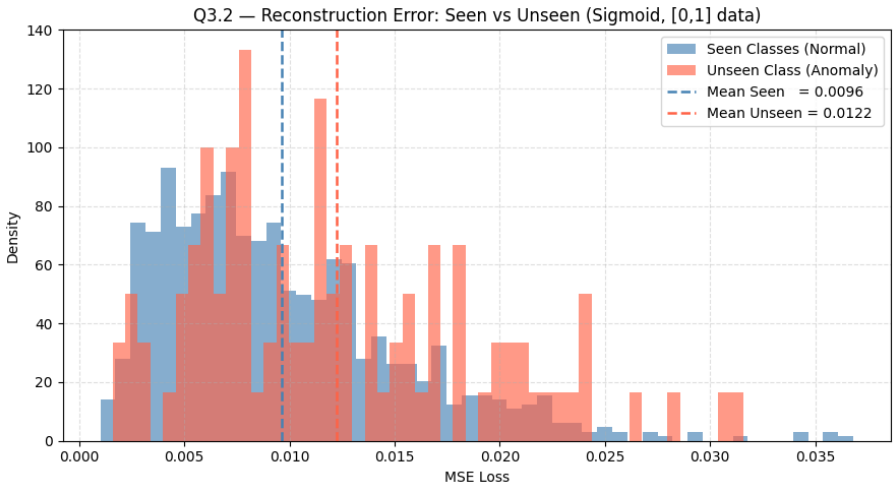

Figure 9: Density histogram comparing reconstruction error distributions for Seen (normal) and Unseen (anomalous) classes using Sigmoid activation.

3.2.3 Analysis

The results indicate that reconstruction error provides a meaningful signal for detecting out-of-distribution samples, although it must be interpreted statistically.

1. **Shift in Error Distribution:** The Unseen class exhibits a higher average reconstruction error compared to the Seen classes, indicating that the Autoencoder struggles to reconstruct samples outside the learned distribution.

2. **Distribution Overlap:** There is noticeable overlap between the two distributions in the low-error region. This suggests that reconstruction error alone is insufficient for perfect separation, particularly when classes share similar low-level visual features such as edges and textures.

3. **High-Error Tail:** The Unseen class contributes more samples in the higher error region, forming a right-skewed tail. These correspond to images with structural characteristics not captured during training.

Overall, the Autoencoder demonstrates sensitivity to distributional shifts by assigning higher reconstruction errors to unseen data. However, due to overlapping error distributions, anomaly detection using reconstruction error is best interpreted statistically rather than deterministically.

This behaviour highlights a key limitation of reconstruction-based anomaly detection: the model is more sensitive to structural deviations than purely semantic differences between classes.

3.2.4 Conclusion

The Autoencoder trained on 9 Seen classes successfully assigns higher reconstruction errors to images from the Unseen class, validating reconstruction error as a viable anomaly score. While the separation is not absolute due to distributional overlap, the consistent shift in mean MSE and the right-skewed tail of the Unseen error distribution confirm that the model has learned a meaningful representation of the Seen class manifold. Any sample that deviates significantly from this manifold incurs a higher reconstruction cost, which can be exploited for anomaly detection in a probabilistic framework.

3.3 Ablation Study: Sigmoid vs. Tanh as Final Activation

3.3.1 Objective

This ablation study investigates which final output activation **Sigmoid** or **Tanh** works better when input pixels are normalised to $[0, 1]$.

3.3.2 Theoretical Analysis

Sigmoid is the theoretically natural choice for $[0, 1]$ normalised inputs:

- Output range $[0, 1]$ aligns exactly with the input range.
- MSE is computed between quantities on the same scale with no post-processing.
- No additional rescaling is required inside `forward()`.

Tanh outputs values in $[-1, 1]$, which introduces a range mismatch:

- The rescaling $(x + 1)/2$ inside `forward()` is mandatory to bring the output back to $[0, 1]$ before computing MSE. Without it, the loss is inflated and the two models are not comparable.
- Tanh has a steeper gradient near zero compared to Sigmoid, which can help convergence for pixel values in the mid-range, this is why Tanh can sometimes achieve a lower absolute MSE despite the added rescaling step.

3.3.3 Results

Table 6: Ablation Study — Average Reconstruction MSE: Sigmoid vs. Tanh

Activation	**Seen MSE**	**Unseen MSE**	**Gap (Δ)**
Tanh (+ rescale)	0.0084	0.0098	0.0014
Sigmoid	0.0096	0.0122	0.0026

3.3.4 Analysis

Empirically, Tanh achieves a lower absolute reconstruction MSE. This is due to its stronger gradient signal in the mid-range of pixel values, allowing faster convergence within 30 epochs. However, for anomaly detection the critical metric is not the absolute MSE but the **gap** Δ:

$$\Delta = \text{MSE}_{\text{unseen}} - \text{MSE}_{\text{seen}}$$

A larger Δ means better separation between normal and anomalous samples. The activation with the larger gap is the more effective anomaly detector, regardless of which achieves the lower absolute error.

3.3.5 Conclusion

When input pixels are normalised to $[0, 1]$, **Sigmoid is the theoretically correct choice** due to its natural range alignment with the input, direct MSE interpretability, and no requirement for rescaling. Tanh can achieve lower absolute reconstruction error due to gradient advantages, but requires mandatory rescaling and does not necessarily produce better anomaly detection. For this task, the activation producing the larger Δ is preferred, which must be verified from the experimental values in Table 6.